

EdgeBrain: A Game Based Collaborative Computational Task Offloading Framework for Edge Video Analytics

Hui Sun¹, Zhiyong Wang², Ying Yu², Kewei Sha², *Senior Member, IEEE*, and Yalong Wu³

Abstract—With the development of the Internet of Everything (IoE), huge amount of data is being generated at network edge and needs to be processed in real time, such as edge video analytics applications. However, limited computing resources and energy on edge devices make it challenging to efficiently perform the intensive computing tasks. Existing solutions of offloading tasks to edge servers depend on the availability of edge servers, and they are difficult to satisfy massive computation needs. Instead, this article proposes a novel offloading framework named EdgeBrain, which offloads computational tasks from one edge device (Producer of Computation, i.e., PCO) to other edge devices (Consumers of Computation, i.e., CCOs). We formulate this offloading problem as a multi-round non-cooperative Stackelberg game, prove the existence of unique Nash Equilibrium (NE) and Stackelberg Equilibrium (SE) in the game, and design the gradient search algorithm to calculate the optimal offloading decision. The performance evaluation based on a prototype implementation of EdgeBrain in the context of a video analytics application, face detection and recognition, shows that EdgeBrain outperforms several comparable techniques in terms of execution time, frame rate, transfer data volume, and power consumption.

Index Terms—Edge computing, video analytics, distributed computational task offloading, non-cooperative Stackelberg game, Nash equilibrium.

I. INTRODUCTION

INTERNET of Everything (IoE) is remodeling the world with pervasive connectivity between heterogeneous networks and intelligence in diverse smart systems [1], [2], [3]. The number of IoE devices is estimated to reach 38.6 billion in 2025 and 50 billion in 2030 [4]. Video analytics is pervasively used in surveillance systems. The video collected by smart edge devices needs to be analyzed either in the cloud or at the edge. Cloud-based video analysis has the advantage of high accuracy because of its enormous storage and computing capacity, but it could also

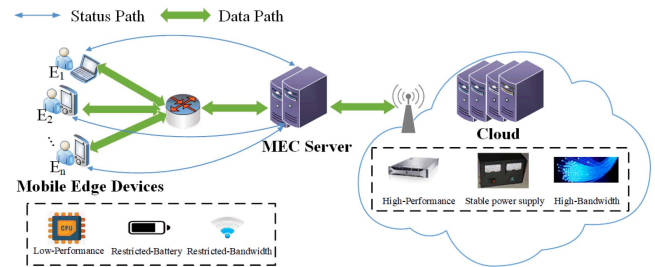


Fig. 1. A typical task offloading framework in MEC.

result in high delay, high bandwidth, and energy consumption in transmission.

Recently, researchers have proposed mobile edge computing (MEC) to relieve the above concerns [5], [6], [7], [8]. In Fig. 1, MEC servers are deployed close to edge devices and provide computing services for edge devices to offload tasks. This approach usually reduces latency and energy consumption, as well as achieves excellent analytical results. Therefore, it is widely used in edge video analytics (EVA) [9], [10], [11], [12]. However, this approach depends on the availability of the edge server and may overwhelm the server with extensive computation. Moreover, the computation-offloading strategies in MEC are mostly single-direction, i.e., the offloading is from edge devices to MEC servers. Therefore, researchers seek alternative solutions. One interesting idea is to offload tasks among edge devices. Recent work such as intelligent mine [13] and intelligent wind farm [14] shows that computational task offloading among edge devices provides lower delay, lower packet loss, and higher bandwidth in IoT systems. In addition, studies [15] have revealed that the CPU usage of a mobile edge device is often less than 33.3%, and there are many remaining computing resources. Therefore, a mobile edge device with many tasks and constrained resources can offload its tasks to adjacent edge devices with idle computing resources instead of to the edge server. This can reduce the edge server's computational overhead and the up-link network traffic to the cloud or the edge server.

To achieve task-offloading among adjacent edge devices, recent research proposes collaborative offloading frameworks in [16], [17]. In these frameworks, the tasks from the producers of computational task offloading (i.e., PCOs) can either be executed locally or offloaded to nearby available edge devices called consumers of computational task offloading (i.e., CCOs). Li

Manuscript received 12 May 2023; revised 9 December 2023; accepted 11 December 2023. Date of publication 14 December 2023; date of current version 9 October 2024. This work was supported in part by the National Natural Science Foundation of China under Grants 61702004 and 62072001. (Corresponding author: Kewei Sha.)

Hui Sun, Zhiyong Wang, and Ying Yu are with the School of Science and Technology, Anhui University, Hefei, Anhui 230093, China (e-mail: sunhui-working@gmail.com; zhyong@ahu.edu.cn; yingyu@ahu.edu.cn).

Kewei Sha is with the Department of Information Sciences, University of North Texas, Denton, TX 76205 USA (e-mail: comersha@gmail.com).

Yalong Wu is with the Department of Computing Sciences, University of Houston–Clear Lake, Houston, TX 77058 USA (e-mail: wuy@uhcl.edu).

Digital Object Identifier 10.1109/TSC.2023.3342800

et al. [18] proposed an online incentive mechanism for collaborative task offloading to utilize global resources. Nonetheless, the partial task offloading strategy is not considered in these approaches. In our work, we apply this strategy to achieve the distribution task offloading, which can process tasks in parallel and fully utilize the resources of nearby devices. To design a distributed edge computing framework, we face the following challenges. *First*, the relationship between a PCO and CCOs is complex when edge devices interconnect and collaborate with each other. Particularly, it is challenging to leverage the cooperative relationship among edge devices when decentralized task offloading occurs. *Second*, when multi-user task offloading requests arrive simultaneously, it is difficult for an offloading strategy to schedule the requests from the PCO under the requirement of users' performance. *Third*, to achieve the success rate of offloading tasks, we need to consider the offloading decision-making, battery on edge devices, and harsh network conditions. In this case, it is crucial to select available CCOs to ensure the computational task offloading.

To tackle the above challenges, work [19] adopts a '0-1' offloading scheme, and the problem of task offloading between end-users and edge service providers is generalized to a repetitive Stackelberg scheme. The authors employed the Stackelberg algorithm to meet the end-user's requirements. However, the '0-1' offloading scheme may lead to low resource utilization when the remaining edge resources are insufficient to support the whole task. One task is divided and offloaded to one remote edge device in the partial offloading scheme. The authors of [20] employed this scheme to build a multi-leader and multi-follower Stackelberg game between MEC servers and end-users (like PCO in our work). In the distributed edge collaborative environment, unlike resource-rich edge servers, edge devices have limited computational resources. The partial offloading scheme rarely considers dividing a task into multiple subtasks that are collaboratively executed on multiple edge devices. When the task requires high computational resources, these two edge devices may fail to offload the task due to insufficient computational resources. In work [21], the Stackelberg game is used to avoid task offloading being affected by the distributed computing resources in the edge server and the dynamic changes in the battery of mobile edge devices. However, this study rarely considers the collaboration between the distributed edge devices with resources during one offloading procedure. The researchers [16] considered an edge collaborative offloading model and designed an online policy encouraging idle edge devices to help resource-limited ones conduct the tasks. In the context of the similar computational capabilities of all edge devices and resource constraints, these idle edge devices cannot complete the task individually.

The differences between our work and the above work can be summarized as follows. First, in this paper, we focus on data partitioned-oriented applications (e.g., recognition and vision applications) [22], where the task can be arbitrarily divided into independent subtasks. Second, in our problem modeling, the tasks are divided into multiple subtasks, which can be parallelly offloaded to multiple suitable edge devices. This not only reduces delay because of parallel processing but also increases

edge resource utilization. Third, in our design, collaborative edge devices provide task-offloading services to others. This can be particularly useful in applications where it is challenging to deploy powerful edge servers. Fourth, our formulated Stackelberg game is different from their games, and therefore, the solutions to task partitioning are also different. In summary, multiple edge devices collaboratively process tasks, improving the resource utilization of edge devices and enhancing the quality of service (QoS).

Thus, we propose a novel non-cooperative Stackelberg game-based computational task offloading framework named EdgeBrain. Our proposed model focuses on a multi-edge computing environment supported by multiple edge service providers (in our case, the CCOs). In EdgeBrain, the edge devices with EVA tasks will act as the PCO to offload computation tasks, while the CCOs will utilize their local available computing resources and network bandwidth to process the tasks with the PCO collaboratively. To maximize the joint utility of PCO and CCOs in EdgeBrain, we formulate it as an ingenuity optimization problem and design an online incentive mechanism based on a non-cooperative Stackelberg game. Specifically, we convert the optimization problem into two steps. In the first step, the PCO determines the optimal payment strategy. In the second step, the CCOs identify the optimal offloading strategy based on the optimal payment strategy. Nonetheless, both steps cannot improve the joint utility of PCO and CCOs by adjusting their respective strategies. Thus, we apply the non-cooperative Stackelberg game to formulate the interaction between the PCO utility and CCO utility, by which we can fully utilize the resources in the MEC system and improve system performance. In a heterogeneous edge computing system, edge devices can provide different energy consumption and time costs per task for applications. Our proposed offloading scheme considers both the energy cost and the time cost of task execution. The algorithm strives to offload tasks to edge devices that generate more revenue to minimize energy and time cost and maximize the system's overall revenue, thereby improving the QoS to process the tasks. Contributions of this paper are summarized as follows:

△ We design a novel task offloading framework comprising multiple edge devices. In the task offloading scheme, we consider the horizontal collaborative capability among edge devices and focus on computing resources, network bandwidth, and battery of each edge device in the distributed edge environment. With the limited computational resources on edge devices, we split the task into multiple subtasks offloaded to neighboring edge devices, improving the QoS for the task execution.

△ To maximize the overall benefit of the system and balance load among edge devices, we propose a new online offloading framework based on the Stackelberg game. In addition, to verify the reliability of the Stackelberg game, we theoretically prove the existence of Nash equilibrium and the uniqueness of the existence.

△ In multi-user-requests scenarios, we design a priority-based scheduling algorithm to offload requests on PCOs based on users' performance requirements and task tolerant time. The mobility of edge devices and harsh networks can fail

computational offloading. Then, we formulate a CCO availability model as the prior condition for candidate CCOs, which guarantees the success rate of computational task offloading.

△ We conduct extensive experiments to validate our collaborative computation strategy based on the Stackelberg game in terms of execution time, frame rate, power consumption on edge devices, and transmission data volume between a PCO and CCOs. The results unveil that EdgeBrain has excellent performance and scalability.

The remainder of this paper is organized as follows. Section II introduces the related work. Section III describes the system model in this work. We present the problem formulation (i.e., Stackelberg game and Nash equilibrium) in Section IV. Section IV-D demonstrates the implementation of the Stackelberg game-based collaborative computational offloading strategy. In Sections VI and VII, we conduct experiments to study the performance of the schemes. Finally, we conclude this paper in Section IX.

II. RELATED WORK

The existing studies on computational task offloading in MEC focus on resource allocation and computational task offloading strategies.

Resource Allocation in Edge Environment: The edge computing paradigm is used to compress video analytics near data sources [23]. Existing EVA on resource allocation have focused on MEC offloading [24], which can be divided into three categories based on the optimization objectives and different scenarios 1) minimizing the task delay [7], [25], 2) minimizing the overall energy consumption [26], [27] and 3) maximizing the global revenue [28].

To reduce the delay, Cloudlet [25] establishes a three-layer architecture - mobile edge devices, Cloudlet, and cloud. ETSI [7] introduces multi-access edge computing (i.e., MEC) to process mobile edge tasks on the wireless access side. At MEC servers, a deep reinforcement learning-based resources allocation strategy [29] is proposed to minimize long-term total execution time and resources cost. Thanks to the limitation of battery and bandwidth at edge devices, an online dynamic task allocation strategy based on an energy harvesting device can balance the energy consumption and delay in the MEC [26]. Mao et al. [30] provide an energy-aware task offloading strategy to optimize CPU frequency and power consumption in mobile task offloading. A mobile edge relay server [27] offloads tasks to edge devices based on a game theory to minimize the weighted sum of energy consumption and delay. The authors of [28] design a hierarchical network architecture and propose an auction-based profit maximization approach.

Collaborative Task Offloading Strategy: The task offloading schemes have been the vital focus in edge computing. In vehicle fog computing, the authors of [31] proposed an energy-efficient dynamic task offloading and resource allocation scheme that reduces energy consumption and latency. In work [19], the problem of task offloading between end-users and edge service providers is generalized to a repetitive Stackelberg scheme, and the authors employed the Stackelberg algorithm to meet the

end-user's requirements. However, the algorithms of [19], [31] employed a '0-1' offloading scheme. Work [20] built a multi-leader and multi-follower Stackelberg game between MEC servers and end-users, and they employed a partial offloading scheme to offload tasks between an end-user and an edge server. However, this scheme makes it challenging to accomplish tasks, affecting the QoS. In the healthcare IoT [32], portable devices are characterized by energy consumption, short battery lifetime, and delay intolerance. The reinforcement learning-based task offloading algorithms minimize the energy consumption and latency in the constraints of end-to-end latency and battery. In the data-driven industrial IoT domain, amounts of data generated by industrial IoT devices cause high power consumption in the cloud and high operational costs [33]. Thus, the authors proposed a heuristic energy- and hotness-aware resource allocation scheme to optimize the computation, communication, and energy consumption in the cloud.

Collaborative task offloading strategies [16], [34] have been studied. A dynamic pricing-based double auction was proposed in [34] to maximize the system efficiency while encouraging mobile devices and edge servers to participate. An online incentive algorithm [16] is developed for distributed collaborative task offloading to maximize the total revenue. To minimize the delay in task offloading, Chen et al. [21] convert the problem of task offloading into a sub-problem of task placement and resources allocation. Zhao et al. [35] propose a joint resource allocation scheme for mmWave MEC to minimize the delay. However, this work rarely considers the task partitioning based on the free resources of the CCOs; however, mobile devices have limited resources and cannot receive a whole offloading task at once. Mobile devices or edge servers cannot provide free resources without any return.

Recent studies focus on the auction incentive model for task offloading in MEC. In work [36], an incentive mechanism is designed to enable the cloud and edge servers to collaboratively conduct tasks, forming the interaction between the cloud and the edge server as in the Stackelberg game. The optimal payment and offloading strategies maximize the utility of cloud service providers and edge servers. A Stackelberg game-based decentralization algorithm in edge systems is designed to balance computational tasks [37]. Work [38] explores sensors' data fusion and cooperation to optimize coverage performance in wireless sensor networks.

When multiple offloading requests exist in distributed edge systems, task producers' priorities should be considered. Zhao et al. [39] propose a priority-based offloading strategy to meet the delay requirements. Low-delay tasks are preferentially offloaded in the local; otherwise, tasks that tolerate high delay are offloaded to the cloud. Thus, MEC can meet users' delay requirements while processing the maximum number of tasks. During task offloading, the harsh network between mobile edge devices and the limited battery of CCOs may fail to offload tasks. Previous work uses the mobility model as a decision-making basis for task offloading. Kilho Lee et al. [40] design a mobile user trajectory model-based offloading strategy to ensure the task offloading. TS-SMOSA [41], a centralized algorithm for multi-task offloading, takes the user's mobile similarity as an

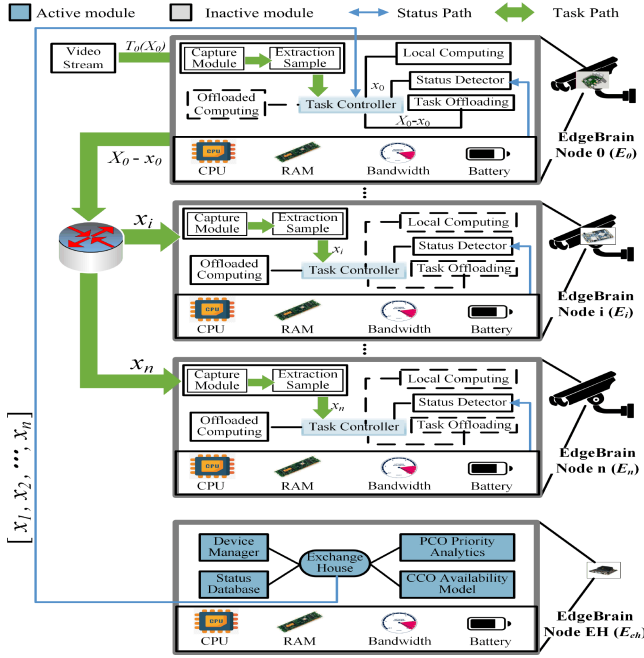


Fig. 2. Exemplified overview system of EdgeBrain.

indicator to determine the task offloading. MAGA [42] proposes a mobile access prediction algorithm based on tail matching sub-sequence according to the user mobility in terms of pattern, periodic, and prediction.

Summary: Unlike the existing works, we consider the potential computing resource of idle mobile edge devices. Then we design a distributed collaborative task offloading framework, which can encourage more CCOs to participate in task processing and optimize computation and battery resources. Besides, to meet the challenge of task offloading in the multi-user scenario, we design the dynamic priority offloading model based on users' subjective demands and objective constraints.

III. SYSTEM OVERVIEW

EdgeBrain consists of a producer of computational task offloading (PCO), a consumer of computational task offloading (CCO), and an exchange house, see Fig. 2. PCO is an edge device that produces offloading tasks and is denoted as E_0 . We can configure PCO according to the priority of video analytic tasks. CCOs, represented as E_i ($i \in \mathcal{N} = \{1, 2, 3, \dots, n\}$), collaboratively process offloaded tasks from E_0 . The exchange house node (E_{eh}) collects status data (i.e., CPU, network and battery of edge devices) to make a decision for offloading tasks. The role of a PCO or CCO can dynamically change according to task priority and edge device reliability. For example, we have a face detection and recognition task $L^{E_0}(X_0)$. L^{E_0} denotes the task on the PCO actor (E_0), and X_0 represents the number of sub-tasks of L^{E_0} . These sub-tasks can be performed locally or collaboratively executed by multiple CCOs. We deploy a Stackelberg game-based algorithm on the exchange house that produces the task offloading strategy according to the status of all edge devices.

PCO and CCO Nodes: In Fig. 2, the *video capture* and *extraction modules* in a PCO generate a video analytic task and send it to the task controller module. Such a module issues a task-offloading request to the exchange house and receives an offloading decision profile. Because of the limited computational resources on edge devices, it is challenging to utilize the resources to support the whole tasks compared to that on resource-sufficient edge servers. Therefore, we propose a strategy to divide a task into multiple subtasks and offload these subtasks to CCOs for parallel execution. Our work aims to improve the QoS of end-user task execution and increase the utilization of computational resources in the distributed edge collaborative environment. For example, E_0 (PCO) initiates a face detection and recognition task (e.g., $L^{E_0}(X_0)$) which is divided into face detection and recognition sub-tasks. The *task controller module* of edge device determines whether to trigger an offloading request according to users' requirements (i.e., delay tolerance) and local resources (i.e., remaining battery). The task is conducted locally when local resources meet the video-analytic requirement; otherwise, the task should be divided and offloaded to CCOs. The PCO sends an offloading request to the exchange house through the *status detector*. The task offloading procedure is triggered according to the offloading decision profile in the exchange house.

CCOs perform sub-tasks offloaded from the PCO while sending information of the devices in real-time to *exchange house*. The *status detector* collects task-offloading requests, CPU cost, memory, bandwidth, and battery of edge devices and sends them to the exchange house. EdgeBrain manages status messages used in decision-making for the offloading strategy. We employ ZAB-BIX [43] to collect the status of edge devices and task offloading requests in MySQL. The task controller receives a task offloading decision-making profile $\mathcal{X} = \{x_1, \dots, x_i, \dots, x_n\}_{i \in \mathcal{N}}$ from the exchange house. x_i denotes a portion of the task named a sub-task on E_i . Task $L^{E_0}(X_0)$ has $(X_0 = \sum_{i=0}^n x_i)$ sub-tasks. Sub-task x_0 ($x_0 = X_0 - \sum_{i=1}^n x_i$) is executed locally, and x_i are offloaded to E_i ($i \in \mathcal{N}$). The *offloaded computing module* receives and performs the sub-tasks from PCO. When resources of CCOs meet the requirement of tasks, the PCO offloads $(X_0 - x_0)$ sub-tasks to CCOs according to the offloading decision profile $\mathcal{X}$.

Exchange House Node: When receiving the request for offloading task, the *exchange house* conducts the Stackelberg game between PCO and CCOs based on the status of all edge devices (see Section IV). Then, we can obtain the offloading decision from this module for the task-offloading request.

The exchange house relies on the *device manager* that manages all edge devices. For a new edge device, the device manager creates a device table and stores its memory, CPU cost in the *status database*. Each device has a database, and a comprehensive database is configured for the system log. The dynamic information includes real-time updating and event-trigger information. The status database stores decision results and the status of the PCO and CCOs. Multiple tasks from more than one PCOs need to be scheduled according to tasks' priority. An edge device returns the result and releases resources after its sub-task completes; meanwhile, the devices' status is updated in

real-time. Multi-user task-offloading requests are featured with different delay and accuracy. We propose a user priority-based scheduling scheme for requests. The PCO priority analytics module stores requests in a queue according to the tasks' priority and waiting time (see Section V-A).

When edge devices involved in the computational task offloading procedure continually move around an area, this procedure can fail as the mobility of edge devices degrades the wireless bandwidth among edge devices. Besides, the less battery on edge devices can also disrupt this procedure. Thus, we formulate a *CCO availability model* to select edge devices as CCOs, which provides a prior condition to determine the optimal offloading and payment strategies. Then, the *exchange house* provides the PCO with an offloading decision profile according to the available CCOs.

IV. PROBLEM FORMULATION AND NASH EQUILIBRIUM ANALYTICS

In EdgeBrain, based on the resources cost of CCOs, the PCO provides CCOs with compensation (payment) as an incentive to incorporate them in collaborative tasks. After performing tasks offloaded from the PCO, CCOs obtain the payment. The maximum system efficiency depends on the optimal balance between the utility of PCO and CCOs. We formulate the interaction between both utilities as the Stackelberg game, where the PCO and CCO are the *leader* and *follower*, respectively. The utility of PCO and CCOs are both positive benefit from offloading or receiving the offloading tasks. The PCO optimizes its payment strategy while CCOs optimize the offloading strategy, jointly maximizing the utility of PCO and CCOs.

A. Utility Model of EdgeBrain

To ensure the compactness of the architecture, we first introduce definitions related to the system models in this study. Afterward, we derive the utility of PCO and CCO as the precursor of the later Stackelberg scheme.

1) *Utility of the PCO*: In this section, we provide definitions and formulas for the system model. Then, we present the PCO utility, which is the revenue from offloading task (i.e., computation cost, delay, and energy consumption). E_0 is the initiator of an offloading task $L^{E_0}(X_0)$. We define the vector form of offloading decision profile as $\mathbf{x} = [x_1, x_2, \dots, x_i, \dots, x_n]$, where $x_i \in \mathcal{X}$ means the number of offloaded tasks on E_i . The amount of local tasks (x_0) is $x_0 = X_0 - \sum_{i=1}^n x_i$, where X_0 is the total amount of offloading tasks. When all tasks are locally performed on E_0 , we have $x_0 = X_0$. As mentioned in work [44], the computation cost of local tasks on E_i is $F(x_i) = e^{\alpha x_i}$, the computation cost of local tasks on PCO is $F(x_0) = e^{\alpha x_0}$, and $F(\mathbf{x}) = F(x_0)$, where rate parameter α ($\alpha > 0$) is constant. Let us suppose that

$$\mathbf{m} = [m_1, m_2, \dots, m_n] \quad (1)$$

which is the set of payment profile for CCOs. Then, the PCO's payment cost for CCOs is

$$Y(\mathbf{x}) = Y(x_1, \dots, x_n) = \sum_{i=1}^n m_i x_i \quad (2)$$

where m_i ($i \in \mathcal{N}$) is the unit price of the offloading task for E_i . The execution time and transmission time of sub-task x_i that the PCO offloads to CCO E_i are given as

$$T_i^{exe} = \beta_1 \frac{x_i}{f_i} \text{ and } T_i^{trans} = \beta_2 \frac{x_i}{\gamma_i}, \quad (3)$$

where constant β_1 and β_2 denote the computational cycles and data volume per sub-task, respectively. f_i is the computing capability of E_i to execute the unit task. γ_i ($\gamma_i > 0$, $i \in \mathcal{N}$) is the network bandwidth between E_0 and E_i . The total delay is

$$T(\mathbf{x}) = T(x_1, x_2, \dots, x_n) = T_0^{exe} + \sum_{i=1}^n T_i^{trans}. \quad (4)$$

Communication Model: We consider the limited network resources of edge devices and utilize the wireless LAN model to allocate channel resources to each edge device and model the uplink transmission time. This will minimize data transmission time, thereby increasing the utility of PCO and CCOs. Because the size of computational results is much smaller than the original offloaded data, the feedback delay can be ignored. Thus, we consider the transmission delay of the data between edge devices. Let us assume that the PCO has N channels allocated to the collaborative CCOs, and the bandwidth of each channel is B . The signal interference occurs when multiple CCOs use the same channel, and the channel interference of E_i is given as

$$I_i = \sum_{j \neq i}^n p_j^{trans} |H_i|^2. \quad (5)$$

The wireless communication between a PCO and a CCO suffers from path loss and small-scale fading [45], [46]. Let $|H_i|^2$ be the channel gain from E_0 to E_i . p_i^{trans} is the transmission power consumption between E_0 and E_i . According to the Shannon-Hartley theorem, the uplink data rate of γ_i is denoted as

$$\gamma_i = B \cdot \log_2 \left(1 + \frac{p_i^{trans} |H_i|^2}{\sigma^2 + I_i} \right), \quad (6)$$

where σ^2 denotes the received noise power at an edge device. Then, we have $p_i^{trans} = \frac{\sigma^2(2^{\frac{\gamma_i}{B}} - 1)}{|H_i|^2}$. We also consider the limited battery of edge devices and model the computational and transmission energy consumption for tasks offloading. This method aims to reduce the overall energy consumption of the system without affecting QoS. The transmission energy consumption for offloading tasks from E_0 to E_i is modelled as

$$J_i = p_i^{trans} \cdot T_i^{trans} = p_i^{trans} \cdot \beta_2 \frac{x_i}{\gamma_i}, i \in \mathcal{N}. \quad (7)$$

The total energy consumption of the offloading task on E_0 is modeled as

$$J(\mathbf{x}) = J(x_1, x_2, \dots, x_n) = \sum_{i=1}^n J_i + p_0^{exe} T_0^{exe} \quad (8)$$

where p_0^{exe} is the computing power consumption of E_0 . For the offloading decision profile, the cost of offloading task L^{E_0} from

E_0 (PCO) is written as

$$C(\mathbf{x}) = P_{cost}(\mathbf{x}) = [\lambda_1 F(\mathbf{x}) + \lambda_2 Y(\mathbf{x}) + \lambda_3 T(\mathbf{x}) + \lambda_4 J(\mathbf{x})] \quad (9)$$

where $\lambda_1, \lambda_2, \lambda_3$, and λ_4 are constant coefficients of the PCO cost. When the payment profile for CCOs is $\mathbf{m}$ (see (1)), the PCO utility is

$$\begin{aligned} U(\mathbf{x}) &= C(0) - C(\mathbf{x}) \\ &= e^{\alpha X_0} + \beta_1 \frac{X_0}{f_0} + \beta_1 \frac{X_0}{f_0} p_0^{exe} - e^{\alpha x_0} \\ &\quad - \sum_{i=1}^n m_i x_i - \beta_1 \frac{x_0}{f_0} \\ &\quad - \beta_1 \frac{x_0}{f_0} p_0^{exe} \sum_{i=1}^n \left(\beta_1 \frac{x_i}{f_i} + \beta_2 \frac{x_i}{\gamma_i} \right) \\ &\quad - \sum_{i=1}^n \left(p_i^{trans} \cdot \beta_2 \frac{x_i}{\gamma_i} \right) \end{aligned} \quad (10)$$

2) *Utility of the CCO*: When a PCO initiates a task-offloading request, each CCO can obtain revenue at the cost of its resources to collaboratively execute offloading tasks. The CCO utility is defined as the difference between the revenue $W_i(x_i)$ and the sunk cost $W_i(0)$. Let E_i be a CCO for E_0 's task-offloading request (L^{E_0}). The unit price for a local task is set to r_i , which is expressed as the minimum acceptable bid price. Similar to work [47], for the offloading decision x_i , the revenue of E_i is given as

$$W_i(x_i) = r_i(\chi_i - x_i) + m_i x_i + \alpha \ln(\chi_i - x_i + 1), \quad (11)$$

where χ_i is the maximum acceptable task of E_i , $\alpha \ln(\chi_i - x_i + 1)$ represents the degree of satisfaction from E_i . Then, the CCO utility for the task is given as

$$\begin{aligned} U_i(x_i) &= W_i(x_i) - W_i(0) \\ &= (m_i - r_i)x_i + \alpha \ln(\chi_i - x_i + 1) - \alpha \ln(\chi_i + 1) \end{aligned} \quad (12)$$

where $(m_i - r_i)x_i$ represents the pure gain obtained by E_i that assists E_0 with subtask x_i processing. χ_i is utilized in (12) by setting $\alpha \ln(\chi_i - x_i + 1) - \alpha \ln(\chi_i + 1)$ to restrict it from receiving too many subtasks. When the number of offloaded subtasks x_i exceeds χ_i , the value of $\alpha \ln(\chi_i - x_i + 1) - \alpha \ln(\chi_i + 1)$ will drop sharply, which decreases the utility of CCO_i ($U_i(x_i)$). Moreover, the task volume x_i offloaded to E_i cannot exceed the maximum number of tasks ($\chi_i + 1$) that can be accepted by E_i .

B. Problem Formulation

We assume that the number of computational tasks on the PCO is X_0 . In this study, we focus on task offloading strategies between the PCO and CCOs instead of the communication among CCOs in distributed EVA. Thus, we suppose that CCOs are independent. The PCO primarily makes the payment strategy, while the offloading strategy for CCOs is updated according to the payment strategy. The offloading strategy affects the PCO payment strategy. To achieve the optimal equilibrium between

PCO and CCOs, the Stackelberg game offers a promising solution to the multi-level decision-making process between the followers (CCOs) and the leader (PCO) [48].

For offloading task L^{E_0} , in the first stage, the default payment profile for the PCO is configured as $\mathbf{m}$ (see (1)). In the second stage, according to the payment profile, the offloading decision profile of CCOs is denoted as

$$\mathbf{x}(\mathbf{m}) = [x_1(m_1), \dots, x_i(m_i), \dots, x_n(m_n)], \quad (13)$$

where $x_i(m_i)$ means that offloading decision x_i can be obtained through m_i according to (21). The game process assumes that the PCO is given the unit price of task offloading. A CCO dynamically adjusts the number of executable tasks according to its resources to obtain the maximum utility, the proof is shown in Section IV-C. Thus, the objective of E_i for an offloading task is given as

$$\begin{aligned} &\max U_i(x_i) \\ \text{s.t. } &C_1 : 0 \leq x_i \leq \chi_i, i \in \mathcal{N} \end{aligned} \quad (14)$$

To maximize its utility under the constraints of computing resources, costs, and energy consumption, the PCO should adjust the payment strategy that is denoted as

$$\begin{aligned} &\max U(\mathbf{m}) \\ \text{s.t. } &C_2 : \underline{m}_i \leq m_i \leq \overline{m}_i, i \in \mathcal{N}; \\ &C_3 : f_i \in (0, f_i^{\max}], i \in \mathcal{N}. \end{aligned} \quad (15)$$

Notations $\underline{m}_i$ and $\overline{m}_i$ are the upper and lower bounds of the unit price m_i , respectively. $f_i^{\max}$ denotes the maximum computing capability of E_i . Let $\mathbf{x}^* = [x_1^*, \dots, x_i^*, \dots, x_n^*]$ be an optimal offloading decision profile of the PCO. $\mathbf{m}^* = [m_1^*, \dots, m_i^*, \dots, m_n^*]$ is the PCO's optimal payment strategy. The above two problems are convex optimization problems. We can obtain the extreme value of $U_i(x_i)$ and $U(\mathbf{m})$, see Section IV-C. The Hessian matrix can determine the extreme value point in a multivariate function in the convex optimisation problem. If the Hessian matrix is positive, the extreme value point is obtained. We transform max problem into min problem:

$$\min_{\mathbf{x}} \tilde{U}_i(x_i) = -U_i(x_i) \text{ s.t. } C_1 \quad (16)$$

$$\min_{\mathbf{m}} \tilde{U}(\mathbf{m}) = -U(\mathbf{m}) \text{ s.t. } C_2 \sim C_3 \quad (17)$$

C. Stackelberg Equilibrium Analytics

The relationship between the PCO's and CCOs' utility is expressed as a two-stage Stackelberg game in Section IV-B. Both PCO and CCOs maximize their utility based on the game of the payment for the offloading task. The game's objective is to obtain optimal payment and offloading strategies. First, we define the Stackelberg equilibrium of the Stackelberg game.

Definition 1. Stackelberg equilibrium: Let $(\mathbf{x}^*, \mathbf{m}^*)$ be the Stackelberg equilibrium solution in the Stackelberg game between the PCO and CCOs. When we have

$$\begin{aligned} &(\mathbf{x}^*, \mathbf{m}^*) = \arg \min_{\mathbf{m} \in \Omega} \tilde{U}(\mathbf{x}^*, \mathbf{m}) \\ \text{s.t. } &C_4 : x_i^* = \arg \min_{x_i \in \Phi} \tilde{U}_i(x_i), \forall i \in \mathcal{N}, \end{aligned}$$

$$\begin{aligned} C_5 : \Phi_i &= \{x_i \mid x_i \in [0, \chi_i]\}, \forall i \in \mathcal{N}, \\ C_6 : \Omega &= \{m_i \mid m_i \in [\underline{m}_i, \overline{m}_i]\}, \forall i \in \mathcal{N}. \end{aligned} \quad (18)$$

Theorem 1: There exists a unique Nash Equilibrium (NE) strategy $x^* = [x_1^*, x_2^*, \dots, x_n^*]$ among the CCOs.

Proof: First, given the function $U_i(x_i)$ is continuous and differentiable. The first-order partial derivative of the function $U_i(x_i)$ concerning x_i can be listed as

$$\frac{\partial \tilde{U}_i}{\partial x_i} = -m_i + r_i + \frac{\alpha}{\chi_i - x_i + 1}, \quad (19)$$

The second-order partial derivative of the function $U_i(x_i)$ concerning x_i is given as

$$\frac{\partial^2 \tilde{U}_i}{\partial x_i^2} = \frac{\alpha}{(\chi_i - x_i + 1)^2}. \quad (20)$$

Because of $x_i \in [0, \chi_i]$ and $\alpha > 0$, we have $\frac{\partial^2 \tilde{U}_i}{\partial x_i^2} > 0$, which means that function $\frac{\partial^2 \tilde{U}_i}{\partial x_i^2} > 0$ is the strictly convex function with respect to x_i . Let $\frac{\partial \tilde{U}_i}{\partial x_i} = 0$, the optimal offloading decision strategy can be shown as

$$x_i^*(m_i) = \begin{cases} 0, & m_i < \underline{m}_i \\ \chi_i + 1 - \frac{\alpha}{m_i - r_i}, & \underline{m}_i \leq m_i \leq \overline{m}_i \\ \chi_i, & m_i > \overline{m}_i \end{cases} \quad (21)$$

CCO_i provides whole resources to execute the offloaded task when the PCO payment for CCO_i satisfies $m_i > \overline{m}_i$. When the PCO payment satisfies $m_i < \underline{m}_i$, CCO_i cannot provide resources to execute the task. Then, the best payment strategy m_i^* is located between $\underline{m}_i$ and $\overline{m}_i$.

Lemma 1: For the optimal payment strategy m_i^* , it satisfies $m_i^* \geq \underline{m}_i \geq r_i$.

Proof: According to (21), the first-order partial derivative of x_i^* with respect to m_i is given as

$$\frac{\partial x_i^*}{\partial m_i} = \frac{\alpha}{(m_i - r_i)^2} > 0, \quad (22)$$

where $x_i^*(m_i)$ varies monotonously with m_i . Let $x_i^*(m_i) = 0$, we can obtain $\underline{m}_i = r_i + \frac{\alpha}{\chi_i + 1} \geq r_i$. Thus, the Lemma 1 has been proved. Besides, let $x_i^*(m_i) = \chi_i$, and we have $\overline{m}_i = r_i + \alpha$. The range of the unit price is $[r_i + \frac{\alpha}{\chi_i + 1}, r_i + \alpha]$.

Then, according to (22), the second-order partial derivative of $x_i^*(m_i^*)$ with respect to m_i is represented as

$$\frac{\partial^2 x_i^*}{\partial m_i^2} = -\frac{2\alpha}{(m_i - r_i)^3}, \quad (23)$$

where we only need to prove $m_i - r_i > 0$. We can prove $\frac{\partial^2 x_i^*}{\partial m_i^2} < 0$. According to (21), we can obtain $m_i - r_i = \chi_i - x_i^* + 1$. If $\chi_i > x_i^*$. We prove that $m_i - r_i > 0$; then, $\frac{\partial^2 x_i^*}{\partial m_i^2}$ is proven as a convex function.

$x_i^*(m_i)$ is the strictly monotone increasing function concerning m_i . $\tilde{U}_i(x_i)$ is the strictly convex function (policy set is convex set) with respect to x_i (see (20)). When a given m_i is

in the payment strategy range $[\underline{m}_i, \overline{m}_i]$, the optimal offloading strategy x_i^* exists and is unique. Thus, Theorem 1 is proven.

Theorem 2: There exists a unique Stackelberg Equilibrium among the PCO and CCOs in our game. For the optimal offloading strategy for CCO_i , the PCO has a unique optimal payment strategy.

Proof: Briefly, we use Ξ and Λ to represent $-\alpha e^{\alpha(X_0 - \sum_{i=1}^n x_i^*)} + \frac{\beta_1}{f_i} + (1 + p_i^{trans}) \frac{\beta_2}{r_i}$ and $\alpha^2 e^{\alpha(X_0 - \sum_{i=1}^n x_i^*)}$. Then, the first-order partial derivative of PCO utility U with respect to the payment strategy m_i is shown as

$$\frac{\partial \tilde{U}}{\partial m_i} = (\Xi + m_i) \cdot \frac{\partial x_i^*}{\partial m_i} + x_i^* \quad (24)$$

Then, the second-order partial derivative of the function $\tilde{U}$ concerning m_i can be represented as

$$\frac{\partial^2 \tilde{U}}{\partial m_i \partial m_j} = \Lambda \cdot \frac{\partial x_i^*}{\partial m_i} \frac{\partial x_j^*}{\partial m_j} = \frac{\Lambda \alpha^2}{(m_i - r_i)^2 (m_j - r_j)^2} \quad (25)$$

$$\begin{aligned} \frac{\partial^2 \tilde{U}}{\partial m_i^2} &= \alpha^2 e^{\alpha(X_0 - \sum_{i=1}^n x_i^*)} \left(\frac{\partial x_i^*}{\partial m_i} \right)^2 \\ &\quad + (\Xi + m_i) \cdot \frac{\partial^2 x_i^*}{\partial m_i^2} + 2 \frac{\partial x_i^*}{\partial m_i} \\ &= \frac{\Lambda \alpha^2}{(m_i - r_i)^4} - \frac{2\alpha(\Xi + r_i)}{(m_i - r_i)^3} \end{aligned} \quad (26)$$

$$\begin{aligned} Hess \tilde{U} &= \nabla^2 \tilde{U} = \begin{pmatrix} \frac{\partial^2 \tilde{U}}{\partial m_1 \partial m_1} & \frac{\partial^2 \tilde{U}}{\partial m_1 \partial m_2} & \cdots & \frac{\partial^2 \tilde{U}}{\partial m_1 \partial m_n} \\ \frac{\partial^2 \tilde{U}}{\partial m_2 \partial m_1} & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ \frac{\partial^2 \tilde{U}}{\partial m_n \partial m_1} & \cdots & \cdots & \frac{\partial^2 \tilde{U}}{\partial m_n \partial m_n} \end{pmatrix} \\ &= Mat_{ri1} + Mat_{ri2} \end{aligned} \quad (27)$$

$$Mat_{ri1} = \begin{pmatrix} \frac{\Lambda \alpha^2}{(m_1 - r_1)^2 (m_1 - r_1)^2} & \cdots & \frac{\Lambda \alpha^2}{(m_1 - r_1)^2 (m_n - r_n)^2} \\ \frac{\Lambda \alpha^2}{(m_2 - r_2)^2 (m_1 - r_1)^2} & & \vdots \\ \vdots & \ddots & \vdots \\ \frac{\Lambda \alpha^2}{(m_n - r_n)^2 (m_1 - r_1)^2} & \cdots & \frac{\Lambda \alpha^2}{(m_n - r_n)^2 (m_n - r_n)^2} \end{pmatrix} \quad (28)$$

$$Mat_{ri2} = \begin{pmatrix} -\frac{2\alpha(\Xi + r_1)}{(m_1 - r_1)^3} & \cdots & 0 \\ 0 & -\frac{2\alpha(\Xi + r_2)}{(m_2 - r_2)^3} & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & -\frac{2\alpha(\Xi + r_n)}{(m_n - r_n)^3} \end{pmatrix} \quad (29)$$

According to convex optimization theory, Mat_{ri1} is a real symmetric matrix; thus, it is a positive definite matrix. We need to prove that Mat_{ri2} is a positive definite matrix.

Lemma 2: Mat_{ri2} is a positive definite matrix.

Proof: We only need to prove $\Xi + r_i < 0$. We can prove that Mat_{ri2} is a positive definite matrix. According to (24),

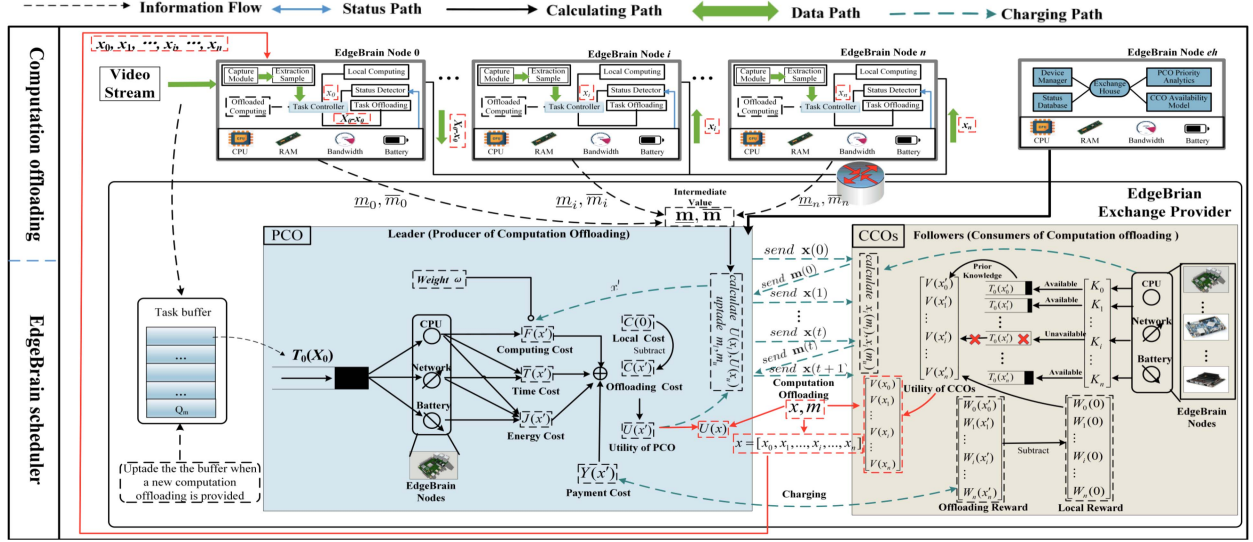


Fig. 3. Data-flow of computational task offloading in EdgeBrain.

let $\frac{\partial \tilde{U}}{\partial m_i} = 0$, the optimal payment strategy m_i^* :

$$\Xi + m_i^* = -x_i^* \cdot \frac{(m_i^* - r_i)^2}{\alpha} < 0 \quad (30)$$

Since $m_i^* \geq r_i$ according to Lemma 1, which means $\Xi + r_i < 0$. Eventually, we can prove Lemma 2, which means the optimal payment strategy $\mathbf{m} = [m_1, m_2, \dots, m_n]$ is unique. It also indicates that the utility function $\tilde{U}(\mathbf{x}^*(\mathbf{m}), \mathbf{m})$ is convex. Finally, the convex optimization problem has been solved, and Theorem 2 is proved. It is reasonable to deploy collaborative tasks to improve the resource utilization and system performance. The above theoretical analytics prove that the computational task offloading strategy based on the Stackelberg game is feasible in EdgeBrain. Thus, we are motivated to design the collaborative offloading framework to achieve optimal offloading and payment strategies.

D. Stackelberg Game-Based Offloading Strategy in EdgeBrain

We formulate the decision-making problem for task offloading between the PCO and CCOs as a two-stage Stackelberg game problem, see Fig. 3. We propose a single-task offloading strategy to obtain the optimal offloading decision after obtaining the available CCOs. Our proposed Stackelberg game-based task offloading is an adaptive strategy. When the edge devices' status or network bandwidth changes, our offloading strategy recalculates and makes a new task offloading decision based on the latest states of edge devices. Before the game is played, the availability-based CCO model (see Section V-B) is used to consider the remaining battery and network bandwidth of the CCOs. Then, we remove some unavailable edge devices and reset the gaming parameters. The status detector collects the status and network bandwidth of PCOs and CCOs in real time and updates the performance indicators such as the maximum number of tasks χ_i that can be received by an edge device and the network resources γ_i that can be allocated to an edge device. Therefore, our offloading strategy adapts to changes in

network bandwidth and edge devices' status, achieving real-time performance, reliability, and robustness.

In a single-task and multi-user scenario, multiple subtasks of a task are offloaded on multiple separate edge devices. For example, face detection and recognition is divided into two subtasks - face detection and face recognition. x_i ($x_i \in \mathcal{X}$) sub-tasks are offloaded to CCO E_i ($i \in \mathcal{N}$) whereas E_0 (PCO) has x_0 offloading tasks. According to the Stackelberg game, we propose Algorithm 1 to execute the task offloading. We use the gradient search algorithm to find the extreme value of a multivariate function. We repeat the calculation of $\mathbf{x}(\mathbf{m})$ and $\mathbf{m}$ according to gradient descent formula $\mathbf{m}^{t+1} = \mathbf{m}^t - \theta \nabla \tilde{U}(\mathbf{m}^t, \mathbf{x}^*(\mathbf{m}^t))$ until the offloading decision $\mathbf{x}$ meets the precision threshold, where θ is the step size of price $\mathbf{m}$ update, $\nabla \tilde{U}(\mathbf{m}^t, \mathbf{x}^*(\mathbf{m}^t))$ is the gradient with $\frac{\partial \tilde{U}}{\partial m_i}$ based on (21) and (24). Finally, we obtain the optimal offloading strategy. In Fig. 3, the exchange house returns the offloading decision profile $\mathcal{X}$ to E_0 and E_i ($\forall i \in \mathcal{N}$).

In the mathematical modeling, we set a series of algorithm parameters for the actual problem. Algorithms 1 repeats the iteration until $\|\mathbf{x}^t - \mathbf{x}^{t-1}\|_2 \leq \delta$, where δ is the accuracy threshold, and $\|x\|_2$ denotes the second-order paradigm of x . The smaller δ is, the higher the accuracy of the result is. The higher the computational paradigm is, the higher the accuracy is, which can cause a high computational complexity. Symbol r_i denotes the computational cost of a unit subtask. Notably, the higher r_i is, the higher the computational cost is. Then, the priority of the receiving task becomes low. r_i is a constant for each CCO. χ_i denotes the maximum number of tasks that E_i can process in parallel. A high χ_i means that more tasks can be processed on E_i in parallel. In addition, the most impact parameter on the model is the hyperparameter α . We have already given a detailed explanation in Section VII-A. A large value of α leads to a high offloading rate. It is worth noting that since our experiments are performed in a prototype implementation instead of simulation, the computational capability of edge devices relates to the CPU frequency and network bandwidth. We studied the

Algorithm 1: Single-Task Offloading Strategy.

Input: precision threshold δ , α , r_i , χ_i , $\forall i \in \mathcal{N}$
Output: $\mathbf{m}^*$, $\mathbf{x}^*$, U , $\mathcal{V} = \{V_1, V_2, \dots, V_n\}$
1: Initializing the input $\mathbf{m}^0 = (m_1^0, m_2^0, \dots, m_n^0)$, where $m_i \leq m_i^0 \leq \bar{m}_i$ according to Lemma 1, number of iterations $t \leftarrow 0$;
2: **while** $\|\mathbf{x}^t - \mathbf{x}^{t-1}\|_2 > \delta$ **do**
3: **for** each CCOs E_i in $\mathcal{E}$ **do**
4: Calculating the amount of sub-tasks x_i^t according to (21);
5: **end for**
6: PCO updates the prices by gradient search algorithm, i.e., $\mathbf{m}^{t+1} = \mathbf{m}^t - \theta \nabla \tilde{U}(\mathbf{m}^t, \mathbf{x}^*(\mathbf{m}^t))$. Where θ is step size of gradient search algorithm;
7: $t \leftarrow t + 1$;
8: **end while**
9: The optimal payment $\mathbf{m}^*$ is obtained;
10: **for** each CCOs E_i in $\mathcal{E}$ **do**
11: Obtain the optimal offloading strategy x_i^* according to (21). The offloading strategy will sent to all participating CCOs;
12: **end for**
13: Obtain utility U and $\mathcal{V} = \{V_1, V_2, \dots, V_n\}$.

computational capability of edge devices in Section III-B. For example, we have $T_i^{exe} = \beta_1 \frac{x_i}{f_i}$ in which β_1 is the physical value, while f_i represents the computational capability of the edge device E_i , see Table I.

E. Convergence of Stackelberg Game-Based Offloading Strategy

To prove the convergence of a Stackelberg game, it is necessary to prove 1) that the optimal response function of the followers exists and is continuous and 2) that the optimal strategy of the leader exists and is unique. Generally, we need to prove that the objective function of the leader's problem is strictly concave or convex. We have proven the two theories in Theorems 1 and 2, respectively. Convergence Analysis: In this section, we analyze the convergence of Algorithm 1. The solutions of the two subproblems can also be applied to the original optimization problem $U(x, m)$. Given $m = m^0$, we can obtain $x = x^0 = x^*(m^0)$ according to the discussion in Theorem 1. In Theorem 2, we refer to $x^*(m)$ as a function with respect to m , and we input $x^*(m)$ into $U(x, m)$. Then, the original optimization problem $U(x^*(m), m)$ can be transformed into an optimization problem $U(m)$ with respect to m . Furthermore, it is worth noting that $x^*(m)$ is a strictly monotonically increasing function with respect to m . We have proven that $U(m)$ is a strictly convex function. We can have the following inequality:

$$U(m^*) > \dots > U(m^t) > U(m^{t-1}) > \dots > U(m^0), \quad (31)$$

where t is the number of iterations. Equation (31) is equivalent to the following inequality:

$$U(x^*, m^*) > \dots > U(x^t, m^t) > U(x^{t-1}, m^{t-1})$$

$$> \dots > U(x^0, m^0). \quad (32)$$

Therefore, $U(x^*, m^*)$ can be obtained by solving the above subproblems and can be used as a valid solution to the original optimization problem.

V. PCO AND CCO SELECTION MODELS

Fig. 3 shows the data flow of the task offloading in Edge-Brain, composed of the uploaded status data and data flow in video analytics. There are four stages - offloading request initiation, CCOs discovery, offloading decision-making, and task offloading. To select a PCO in multi-user scenarios, we model the determination of PCO priority (see Section V-A). We also propose a CCO available model to find CCOs in offloading tasks (see Section V-B). These models help us to make Stackelberg game-based offloading decisions.

A. Priority-Collaborative PCO Scheduling Model

In EdgeBrain, each edge device can act as a PCO. When the exchange house processes a task-offloading request or waits for available resources, a new offloading request may arrive. The scheduler uses a first come first serve (FCFS) policy to schedule requests, but users' performance requirements are different. Thus, we propose a user priority-based scheduling method for task-offloading requests, and define the priority level of PCOs.

Definition 2: Let us have r task-offloading requests from PCOs with different performance requirements. Given E_i 's task $L_j^{E_i}$ ($i \in \mathcal{R}, j \in \mathcal{L}$), let set $\mathcal{R} = \{0, 1, 2, \dots, r\}$ and set $\mathcal{L} = \{1, 2, \dots, l\}$, l be the number of tasks on the PCO. The user priority of $L_j^{E_i}$ depends on (1) the user's subjective demand for computational task offloading and (2) the objective constraint of the computational task offloading. The former, including performance requirements of delay and energy consumption, is denoted as $SD_{L_j^{E_i}}$. The latter includes the amount of offloading/feedback data, which is $OD_{L_j^{E_i}}$. Then, we define the user priority of the offloading sub-task $L_j^{E_i}$ as

$$D_{L_j^{E_i}} = \xi \frac{\sum_{j=1}^l SD_{L_j^{E_i}}}{SD_{L_j^{E_i}}} + (1 - \xi) \frac{OD_{L_j^{E_i}}}{\sum_{j=1}^l OD_{L_j^{E_i}}},$$

$$0 \leq \xi \leq 1.$$

$$SD_{L_j^{E_i}} = D_{\text{latency}} + D_{\text{power}} \quad (33)$$

where ξ is large, the tasks with a slightly tolerable delay are given priority scheduling. Otherwise, the tasks with large data volume will be given a high user priority. We set a queue for offloading requests in the exchange house. The priority of offloading tasks decreases from the head to the tail in the queue. When a new request enters the queue, the exchange house calculates the task priority by Definition 2. The task buffer is updated. Then, the request is inserted into the waiting queue in order of priority. By Algorithm 2, we select the next offloading request according to the priority-collaborative PCO model. When the exchange house determines the offloading request, the PCO is acted as the *leader* of the Stackelberg game, and available CCOs are *followers*.

TABLE I
FEATURES OF EDGE DEVICES USED IN EDGEBRAIN

Device	Raspberry Pi(4B)	Raspberry Pi(3B)	Raspberry Pi(3B+)	Nano PC-T4	Firefly RK3399Pro	Jetson nano	Jetson TX2	ASUSPRO
CPU	ARMv71*4 @1.5	Cortex A53*4 @1.4	ARMv71*4 @1.4	S5P6818*6 @1.4	(Cortex A72*2 +A53*4)@1.8	(ARMv8*4+128-core Maxwell)@1.43	(ARMv8*6+NVIDIA Pascal GPU)@2.0	Intel Core i5-7400*4@3.0
RAM	4GB	1GB	1GB	1GB	2GB	4GB	8GB	12GB
OS ¹	Linux5.4	Linux4.14	Linux4.19	Linux4.4	Linux4.4	Linux4.9-tegra	Linux4.4-tegra	Linux4.15
Label	D0, D3, D9, D10, D11, D12	D1	D2	D4	D5, D8	D6	D7	Exchange House
f_i	0.16684	0.09637	0.11537	0.02217	0.02527	0.15115	0.24861	-

OS: operation systems. Jetson TX2 is powered by a 256-core NVIDIA PascalTM GPU architecture.

Algorithm 2: PCO Selection.

Input: PCO buffer, $T_{wait}^j, T_{offloading}^j$
 $D_{L_j^{E_i}}, i \in \mathcal{N}, j \in \mathcal{L}$

Output: next PCO to be executed

- 1: **for** PCO in PCO buffer **do**
- 2: **if** $T_{wait}^j + T_{offloading}^j \geq \dot{T}$ **then**
- 3: Select the j -th PCO
- 4: **break**
- 5: **end if**
- 6: Select the PCO with the largest $D_{L_j^{E_i}}$
- 7: **if** multiple PCO with the same $D_{L_j^{E_i}}$ **then**
- 8: Select the PCO with the largest T_{wait}
- 9: **end if**
- 10: **end for**

VI. EXPERIMENTAL SETUP

To validate the Stackelberg game-based task offloading strategy, we build a realistic EdgeBrain-enabled distributed EVA that runs face detection and recognition.

Platform: Table I lists eight types of 12 edge devices with operating systems used in EdgeBrain. The programming environment includes Python 3.6, numpy 1.14.5, opencv-python 3.4.2, TensorFlow 1.14.0, scipy 1.2.1, and scikit-learn 0.19.2. We configured these edge devices with Ubuntu 18.04 LTS. The task computation model for each edge device follows the $T_i^{exe} = \beta_1 \frac{x_i}{f_i}$, where $\beta_1 = 1$ represents the computational cycles, and f_i is the computing capability of E_i to execute the unit task. f_i of each edge device is listed in Table I. Their different CPUs, RAM, GPU, network bandwidth, and battery affect the computational task offloading decision-making. Edge devices transmit data through a wired/wireless network connection. Each device can be used as a PCO or a CCO. ASUSPRO being an exchange actor collects the status of resources, network bandwidth, and battery of devices. The status of edge devices inputs Algorithms 1. We then obtain the offloading decision-making and payment strategy for the request of task offloading on the PCO. We formulate the interaction between a PCO and CCOs as a Stackelberg game to maximize the utility, see (14) and (15).

Tested Method: We conduct MTCNN and Facenet for face detection and recognition in EdgeBrain. The dataset includes 2,000 video frames, in which eight-face images are referred to as the task offloading unit. We employ Wondershaper (<https://github.com/magnific0/wondershaper>) to control an edge device's bandwidth: 50 kbps, 100 kbps, 250 kbps, 500 kbps, and 2 Mbps. We use 4 PCOs and 12 CCOs to validate the scalability of the decision scheme. When a PCO initiates a request for task offloading, available CCOs obtain the collaborative rate of face recognition according to the optimal offloading decision profile produced by the multi-round non-cooperative Stackelberg game.

Alternative Schemes: We study seven schemes in this test. In *local scheme*, tasks initiated by the PCO are executed locally. In *cloud scheme*, the PCO offloads tasks to the cloud without PCO-and-CCOs collaboration. *P-offloading strategy*. To achieve the highest PCO utility, we overlook the cost of CCOs in task offloading and the impact of local tasks on performance [51]. In *fine-grained EdgeBrain scheme* (i.e., EdgeBrain), our proposed Stackelberg game-based distributed offloading strategy aims to jointly maximize the utility of the PCO and CCOs for collaborative computation. We simplify the EdgeBrain scheme into *coarse-grained scheme* (i.e., 0-1 offloading). To achieve high PCO utility, the task is executed locally or offloaded to a

B. Availability-Based CCO Model

The collaborative decision-making process between the PCO and CCOs is a multi-round Stackelberg game. Applying many CCOs complicates this game. Thus, owing to the resource constraints of the CCOs, it is challenging to select available CCOs to improve the feasibility and efficiency of task offloading in realistic scenarios before the Stackelberg game executes. According to prior work [49], we apply the network availability between the PCO and CCOs as a CCO availability metric. Besides, CCOs with less battery cannot complete the task, which impairs the success ratio of computational task offloading. In [50], whether a sensor can be selected depends on its capability and contribution. When receiving a task-offloading request, the exchange house removes edge devices with less battery or low bandwidth, which simplifies the Stackelberg game and improves the success rate of offloading tasks.

Definition 3: Given the network availability of E_i as $\eta_i = \frac{MTBF}{MTBF + MTTR}$, the battery of E_i is R_i and the battery required to complete the offloading task is EC_i . Then, the CCO availability is $K_i = \eta_i \frac{R_i - EC_i}{R_i}$, where MTBF is the mean time between two sequential failures, and MTTR is the mean time to repair the failure.

The CCO availability is a prior condition participating in the Stackelberg game, see $\{K_1, K_2, \dots, K_i, \dots, K_n\}$ in Fig. 3. When the edge device's availability is lower than the threshold, it cannot perform tasks and the exchange house cannot select it as a CCO.

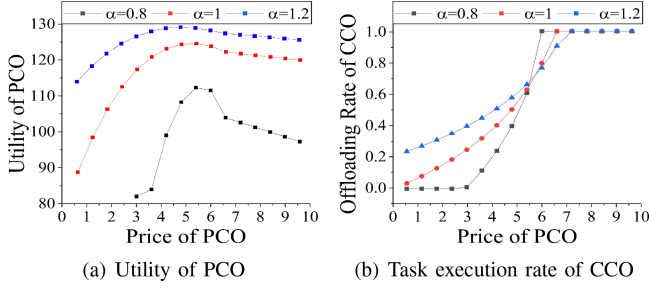


Fig. 4. PCO utility and the task execution rate of CCOs for offloading tasks with PCO unit price.

CCO. *OnlineSchedule* scheme [16] is an online incentive mechanism for collaborative task offloading. With marginal effect rules of resources, *OnlineSchedule* postpones the transmission and execution time of offloading tasks within the users' tolerance, maximizing the overall system utility. We implement a slimmed version as a comparison solution to EdgeBrain. *DPDA* scheme [34] is a dynamic pricing-based double auction mechanism. Because the offloading task is an atomic operation, DPDA matches all requesters and collaborators that offload tasks based on bids. *JPORA* scheme [52] is a D2D-enabled MEC framework, which utilizes the computational resources of near-edge devices to accelerate the local tasks execution. JPORA adopts a partial offloading strategy to provide a dynamic task partitioning scheme and allocation of network spectrum resources. To minimize the total task latency, JPORA jointly considers local energy consumption and task completion deadlines.

VII. EXPERIMENTAL EVALUATION

We provided a system analysis primer, and conducted extensive tests in a realistic application.

A. System Analysis Primer

In this section, we explore the system utility and features of PCO and CCOs used in EdgeBrain.

Utility Analytics: Based on PCO unit price and α in (10), we study the PCO utility and numbers of tasks on the CCO. In Fig. 4(a), the CCO performs fewer tasks when the PCO unit is low, reducing the PCO utility. As the PCO has a high unit price, the CCO performs more tasks. The PCO achieves the highest utility using payment but excessive payment reduces the CCO utility, see Lemma 3. Besides, α affects the PCO utility. Fig. 4(b) shows that the number of offloading tasks on CCOs increases as the unit price becomes large, see Lemma 2. When m_i is decided, the CCO has a unique optimal offloading strategy x_i^* . With a small α , the number of tasks on the CCO varies with the unit price. When $\alpha = 0.8$ and $p < 3.6$, the task offloading rate of the CCO is 0. With a large unit price, this value reaches its maximum. This is because a CCO has a low cost to conduct tasks when α is small. Then, the minimum unit price of the PCO becomes large. However, the maximum unit price is independent of tasks on the CCO, see Lemma 1. Compared with $\alpha = 0.8$ and $\alpha = 1.2$, the curves of the utility of a PCO and the offloading rate of a CCO

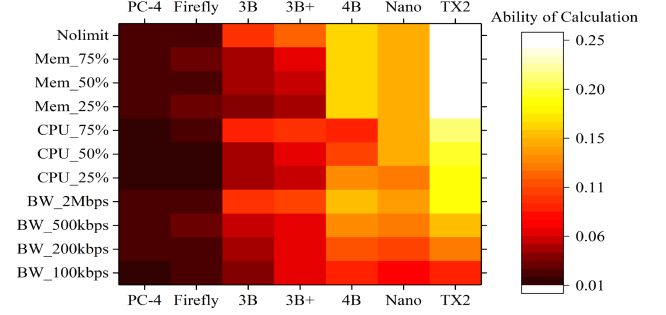


Fig. 5. Average detection frame rate on edge devices.

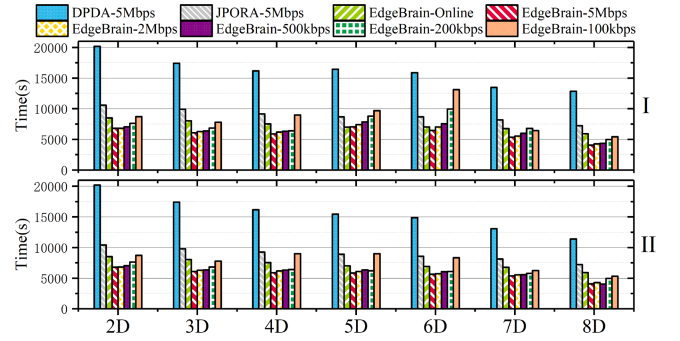


Fig. 6. Execution time varies with CCOs.

are smoother at $\alpha = 1$, which facilitates obtaining the optimal offloading decision point in the game.

Device Analysis: Fig. 5 shows the computational capability of edge devices used in EdgeBrain. We conduct face detection using MTCNN and apply FaceNet to perform face recognition on the platform with different configuration parameters. We test the computational capabilities of each device by fixing other variables. The *whiter block* means that the edge device's computational capability is higher. TX2 with a GPU has the highest computational capability than other devices. Nano and 4B are the second ones, followed by 3B+ and 3B. Firefly and PC-T4 provide the lowest computational capability. Besides, the computational capability of an edge device varies with configuration parameters.

B. Experimental Study

We study the impact of candidate CCOs on the single-task performance of the PCO under 100-kbps, 200-kbps, 500-kbps, 2-Mbps, and 5-Mbps network bandwidths.

1) System Efficiency: Figs. 6 and 7 show the execution time and detection frame rate on a PCO, respectively. Test item iD means that there are i available CCOs in the test. Edge devices D0 and D1 are used in test item 2D. We add D2–D7 sequentially in test items 3D–8D.

We study the impact of the number of CCOs and bandwidth on total execution time. For each CCO, the bandwidth decreases from 5 Mbps to 100 kbps, while two and eight CCOs are configured for a PCO. In Fig. 6, the total execution time increases as the bandwidth becomes low but reduces gradually with more CCOs. However, test items 5D and 6D have higher total execution than

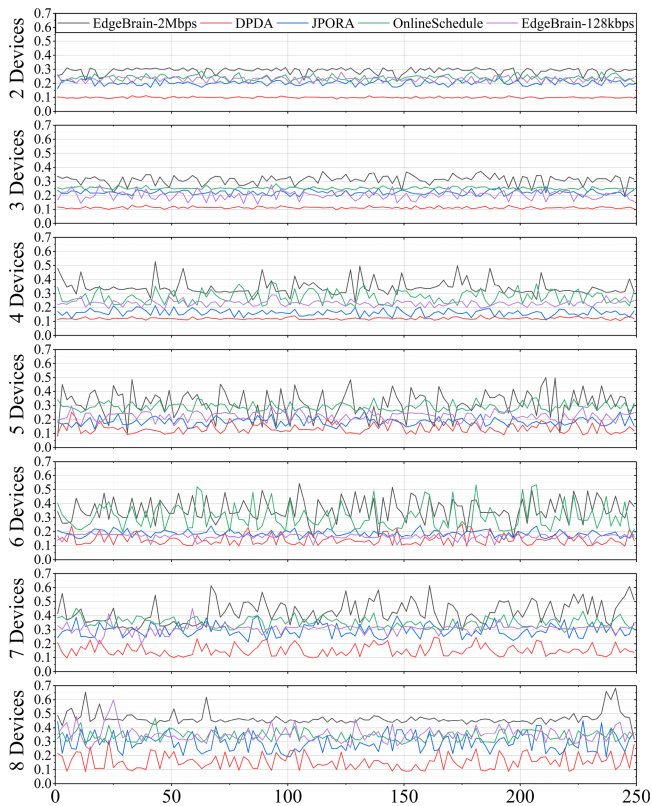


Fig. 7. Detection frame rate varies with CCOs.

4D, see Fig. 6(I) because more CCOs transmit status data and more tasks on the network, causing a high delay. Owing to their low computational capability, the performance of D4 and D5 becomes lower than that of other CCOs in test items 5D and 6D. The execution time is reduced in test items 7D and 8D by using D6 and D7 with high computational capability. In Fig. 6(II), we replace D4 and D5 with D2 and D3. Compared with Fig. 6(I), items 5D and 6D in Fig. 6(II) have lower total time because high-computational capability devices improve system efficiency. DPDA, JPORA, and OnlineSchedule have longer execution time than EdgeBrain because DPDA is an offloading strategy for atomic tasks. In a PCO scenario, the PCO executes tasks locally or merely offloads them to one CCO. By employing time slices to divide tasks, OnlineSchedule executes the tasks locally or offloads them to CCOs. However, to maximize system revenue, OnlineSchedule postpones procedures for task offloading and execution. JPORA has a longer total execution time than EdgeBrain but shorter than DPDA. This is because JPORA selects one remote device at a time for partial task offloading, under-utilizing the computational resources of the remaining edge devices.

Fig. 7 shows the detection frame rate based on various CCOs. To study the performance of EdgeBrain under harsh networks, we configure the detection frame rate at 2 Mbps and 100 kbps. We also study the detection frame rate of DPDA, JPORA, OnlineSchedule, and EdgeBrain under a 2-Mbps network. We listed these findings as follows.

△ Each detection frame rate presents fluctuation. This is because the computational capability of each CCO is heterogeneous, and the CCO's resource dynamically changes when performing tasks. The offloading decision-making in each scheme is changeable. According to offloading decisions, the PCO offloads tasks to each CCO. There is a difference between the Stackelberg game-based offloading decision and the actual optimal one due to the complex environment on a real-world platform, which also exists in other schemes. The slight difference smooths the detection frame rate.

△ Under 100-kbps bandwidth, the detection frame rate of EdgeBrain presents a slight fluctuation. The task offloading is blocked when the bandwidth cannot meet the requirement; then, tasks are executed locally. The CCO still has resources to process tasks offloaded from the PCO. The computational offloading becomes regular, and the sub-tasks are executed in time once they are offloaded to the CCO. Then, the detection frame rate becomes stable.

△ In test items 5D and 6D, the performance of all schemes obviously changes because the computational capability of two new CCOs is low. When network resources are abundant, the execution time of sub-tasks in 5D and 6D is longer than the average time, which is likely to have a trough. Test item 7D employs D7 with high computational capability. When sub-tasks are offloaded to D7 under abundant network resources, the execution time is higher than the average, triggering the fluctuation.

△ Under the 2-Mbps network, the performance of DPDA has less fluctuation than EdgeBrain in tests 2D, 3D, and 4D; however, the detection frame rate becomes low. The reason is that (1) edge devices have similar performance, and (2) DPDA falls into the local optimum. Similar to DPDA, in the case of only one PCO, JPORA only selects the most suitable CCO as the offload object when it initiates task offloading, and there is no need to consider the allocation of communication resources. Still, the difference is that JPORA keeps part of the tasks to be executed locally, so the fluctuation is much more minor than EdgeBrain.

△ The performance fluctuation of EdgeBrain becomes small in test item 8D because D6 and D7 have high computational capability to perform more tasks in the game but D4 and D5 process few or no tasks.

2) *Energy Consumption and Network Transmission:* In Fig. 8, the number of sub-tasks offloaded to each CCO reduces as the number of CCOs increases. Thus, the energy consumption of devices decreases, whereas CCOs have the same trend. The total energy consumption of DPDA, OnlineSchedule, and EdgeBrain becomes low under high-bandwidth networks. Fig. 9 shows that the low bandwidth reduces the data volume from the PCO to each CCO. This is because the low bandwidth of CCOs decreases the PCO utility, affecting the offloading decision-making, see (10). Compared with EdgeBrain, a CCO's atomic tasks offloaded by the DPDA cost higher bandwidth. OnlineSchedule largely depends on the bandwidth of an edge device to find the optimal offloading strategy. DPDA offloads more tasks than EdgeBrain because its task is atomic and PCO has limited computing resources. Notably, EdgeBrain and OnlineSchedule are P-offload strategies that process some data locally. OnlineSchedule

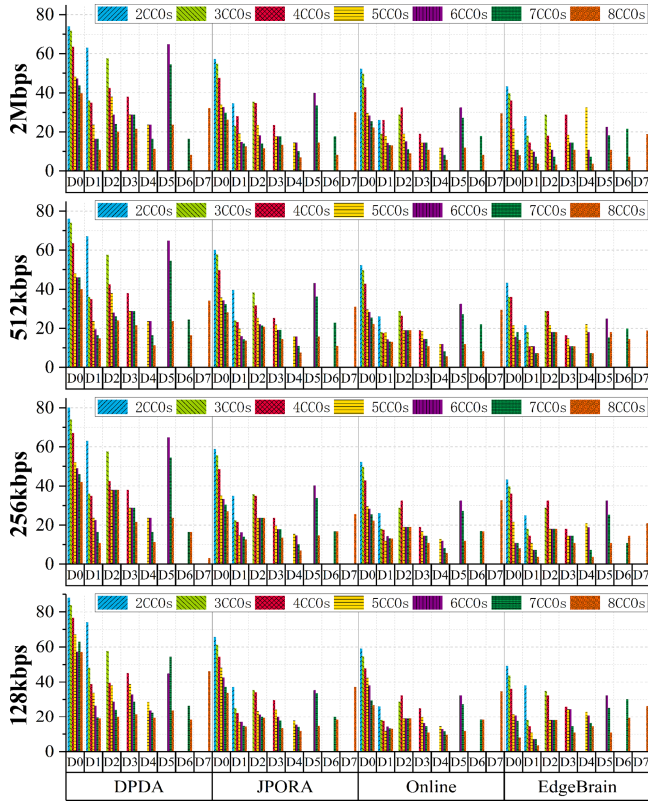


Fig. 8. y -axis denotes total energy consumption (KJ) varies with CCOs and network bandwidths.

deferring tasks slightly enlarges the offloading rate and standby time. Then, DPDA and OnlineSchedule transfer more data than EdgeBrain.

C. Task Offloading Ratio and Execution Frame Rate Study

We study the impact of network bandwidth and the battery of CCOs on computational task offloading rate and execution frame rate under a 1,000-frames video stream.

Impact of Network Bandwidth: To achieve the effect of dynamic CCO availability, we configure 500-, 200-, 100-, and 50-kbps bandwidth on D2 when processing 200th, 400th, 600th, and 800th video frames. The initial bandwidth is 2 Mbps. The bandwidth is considered in the decision; therefore, the offloading ratios of EdgeBrain present slight changes. The local scheme executes all tasks locally, whereas the cloud scheme offloads all tasks to the cloud; thus, the value is zero and one in the two schemes. OnlineSchedule has a smaller offloading ratio than EdgeBrain and DPDA. Besides, the marginal price of network resources on D2 increases as the bandwidth decreases, enlarging PCO's payment. In DPDA, the low bandwidth enhances the marginal price of D2 and bid. Other CCOs with an available network are likely to win the bid; then, the offloading rate from D0 to D2 drops. EdgeBrain has a higher offloading rate than OnlineSchedule and DPDA because sub-tasks are allocated to D2 according to the bandwidth.

In Fig. 10(a), EdgeBrain has the highest performance but its average detection frame rates in the five stages are similar.

The detection frame rate smoothly changes in p-offloading but becomes stable (0.24 fps) in the local scheme. D0 releases resources as task pre-processing completes. The 0-1 offload and cloud schemes reduce the execution frame rate to 0.17 fps and 0.072 fps, respectively. OnlineSchedule is much adept than DPDA, but inferior to EdgeBrain under the high bandwidth. OnlineSchedule is a payment-oriented offloading strategy. To reduce PCO's payment, the task is deferred within limitation. EdgeBrain achieves stable performance by adjusting the offloading decision even though the bandwidth of CCOs is low.

Impact of the Battery of CCOs: We study the impact of CCO availability on the computational task offloading rate. We use the remaining battery of CCOs to define its availability. When processing 250th, 500th, and 750th video frames, we set the battery of D1, D2, and D3 to zero. The task-offloading rate of a PCO varies with unavailable CCOs in task-offloading strategies. In Fig. 10(b), the offloading rate of the five schemes and the cloud scheme is not affected by the CCO availability. The offloading rate of EdgeBrain is 0.636, 0.26, 0.105, and 0 when we use 4 CCOs, 3 CCOs, 2 CCOs, and 1 CCO, respectively. *Few available CCOs* lower the offloading rate because EdgeBrain adjusts the offloading decision profile with the CCO availability. However, in the p-offloading and 0-1 offloading strategies, the offloading rate is zero when D2 is unavailable because the two strategies rarely consider the impact of CCO availability on the success rate of offloading. When the battery of D2 is exhausted, the offloading fails. Thus, the detection frame rates of the two schemes reduce to zero after 250 frames. EdgeBrain has a stable offloading decision. For example, the execution frame rate is in the range of [0.29, 0.35] fps. This is because its status detector makes a prior judgment on the CCO availability. Then, decision-making on available CCOs has little impact on the performance.

D. Extended Study

In this section, we explore the task execution performance and the offloading distribution of EVA in the multi-PCO and multi-CCO scenarios. We employ a 2,000-frame video stream, 4 PCOs, and 12 CCOs in this test.

Total Execution Time: Fig. 11 shows the total execution time when we use 1 - 4 PCOs and 4 - 12 CCOs in EdgeBrain. Because DPDA is superior in the multi-PCO and multi-CCO scenarios, we select it as the baseline. In a single-PCO scenario, the total execution time of test items 5D and 6D increases due to low computational capability of D4 and D5. In a multi-PCO scenario, the total execution time becomes smooth as the number of CCOs increases. This is because EdgeBrain achieves a good trade-off between the PCO cost and CCOs revenues. When the number of CCOs increases, the performance is saturated, and the PCO costs reduce. DPDA is designed for multiple PCOs and CCOs. The execution time increases slowly with the increment of CCOs. This is because DPDA cannot fully exploit the resources of all edge devices when there are few PCOs. The execution time depends on resources on a device instead of distributing tasks reasonably as EdgeBrain. Thus, EdgeBrain has a smaller execution time than DPDA.

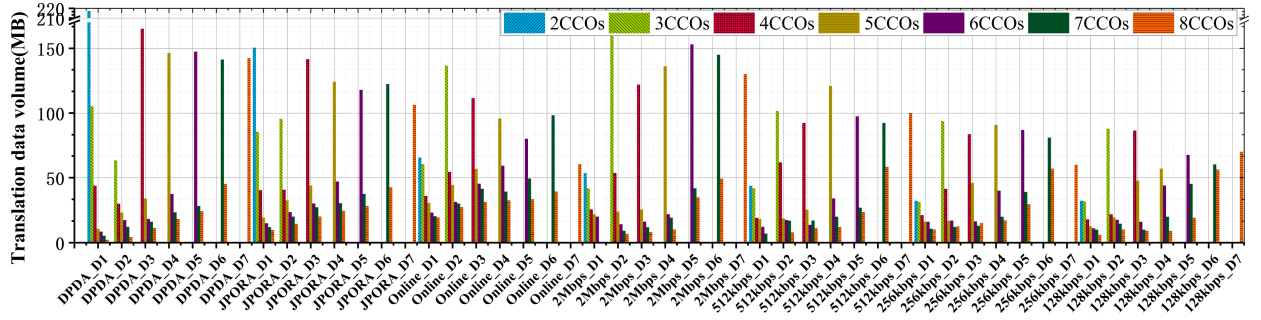


Fig. 9. Transmission data volume varies with CCOs.

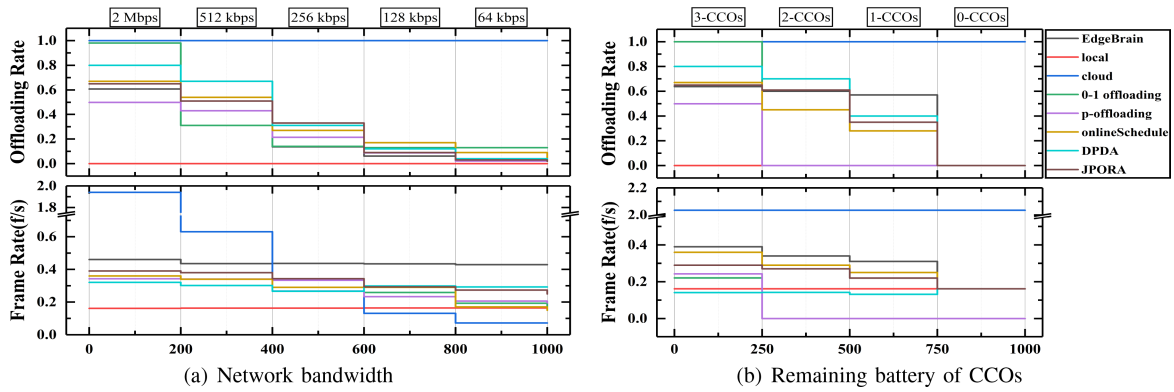


Fig. 10. Computational offloading rate and detection frame rate under network bandwidth and battery on CCOs.

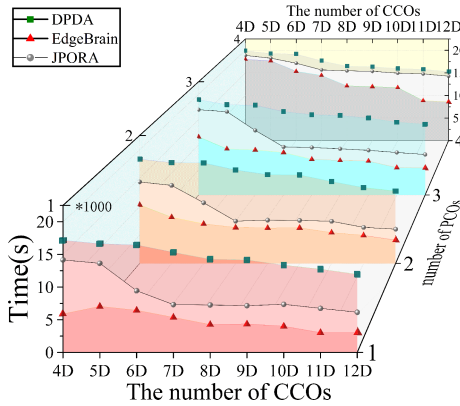


Fig. 11. Total execution time of task offloading using multiple PCOs and CCOs.

Offloading Decision Profile: Fig. 12 presents the distribution of the task offloading decision-making in EdgeBrain. Symbols ip and jc in the x -axis mean the test with the number i of PCOs and the number j of CCOs, respectively. The y -axis denotes 12 edge devices. Each component of the matrix color block denotes a task offloading rate. In test items 1p-4c, 1p-5c, and 1p-6c, the offloading rate on D4 is higher than 40%. In 1p-7c, this value is 25.25%. When there are more than eight CCOs, the offloading rate of D4 is lower than 20%. With more available CCOs, the offloading rate on collaborative edge devices reduces. The high-computational capabilities D7 and D8 are used in test 7- and 8-CCOs. Thus, the offloading rate of D4 sharply

decreases. For example, in test 1-PCO, D5 cannot offload tasks because of its low computing resources. However, in tests with 4 PCOs, D5 offloads tasks because many PCOs need more CCOs to execute tasks collaboratively. The offloading rate of each CCO varies as the number of PCOs and CCOs increases. When the number of CCOs reaches 11, D5 does not participate in collaborative computation. This is because the system performance is saturated with more CCOs, which also exist in D2, D6, and D9.

E. Scalability Analysis

We analyze the scalability of EdgeBrain in terms of network bandwidth, remaining battery, and the number of edge devices.

- **Network bandwidth:** We configured different network bandwidths to study the system performance in the context of sudden changes in the network environment. It is worth noting that the performance of EdgeBrain remains robust even though the network environment deteriorates under a low-bandwidth network. The result demonstrates that EdgeBrain is highly resilient to dynamic changes in the network environment.

- **Battery:** We further studied the impact of battery on the edge device on the performance of the EdgeBrain. Although the gradual depletion of the remaining battery on the edge device impairs the overall performance of EdgeBrain, the performance degradation is relatively slow thanks to the advantages of distributed task offloading strategy and availability-based CCO module.

- **Numbers of edge devices:** We study the impact of the number of edge devices on the overall robustness of the system.

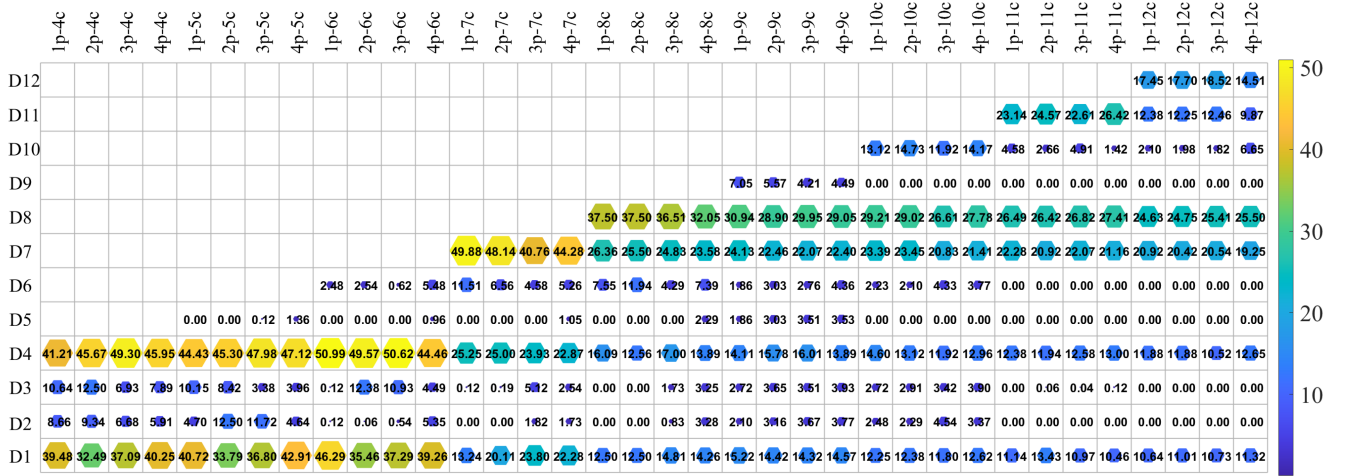


Fig. 12. Offloading rate distribution in EdgeBrain.

First, EdgeBrain provides mechanisms to improve its efficiency and robustness. In Section V-A, we introduce the priority-collaborative PCO scheduling model for multi-user scenarios and select the suitable PCO to offload tasks. Similarly, we design the availability-based CCO model to discover available CCOs before performing offloading decisions, see Section V-B. Second, to validate the scalability of EdgeBrain, we investigate the total execution time when executing the same workload as the number of CCOs and PCOs increases, see Sections VII-B and VII-D. The results show that increasing the number of CCOs does not always improve QoS. This is because the QoS also depends on the performance of the CCOs. On the contrary, increasing the number of PCOs increases the system's load and degrades the system performance. Compared with other offloading schemes, EdgeBrain can better handle tasks with multiple PCOs due to the parallelism of its distributed task offloading, which improves the efficiency of task processing. Notably, a PCO offloads tasks to neighboring CCOs and the number of CCOs will not be too large. Therefore, we may not focus on the number of edge devices in this study.

EdgeBrain shows much robustness in terms of task offloading rate and video detection frame rate than alternative schemes. These results validate the robustness and flexibility of EdgeBrain in the face of device resource variations.

VIII. LIMITATION AND FUTURE RESEARCH DIRECTIONS

In EdgeBrain, there are still some limitations in terms of the order of task offloading, failure tolerance, and task offloading time. We present the research directions to improve the limitations in the future.

First, in EdgeBrain, we used priority queues to schedule task offloading requests initiated by multiple PCOs. However, we rarely consider the order of processing task offloading requests in this study. Taking global order of these requests into consideration may further improve the system's performance. In future work, we will rebuild the model and extend this study to the scenario of multi-PCO and multi-CCO to improve the

global system performance. We will comprehensively consider the requirements of all tasks and design global optimization algorithms on task scheduling, resource allocation, and load balancing, improving the system efficiency. When the load is high, new offloading requests can be queued to ensure the execution of existing tasks.

Second, in realistic, task-offload failures can be an unpreventable issue; therefore, a sound mechanism is needed to tolerate task-offloading failures. In future work, for the task offloading failures, some small-sized tasks with dependencies will be merged by EdgeBrain to reduce the complexity of task scheduling and improve the success rate. We will design a fault-tolerant mechanism in the future. When task offloading fails, the system can automatically detect and retry to offload the task to other available CCOs. This mechanism improves the system's robustness and ensures that tasks can be executed promptly after failure.

Third, tasks deployed on the PCO have different requirements for real-time. According to the CCO's available resources (e.g., network), choosing the appropriate task offloading time can improve the system performance and QoS. In future work, to determine the best time to offload tasks, we will use historical data and machine learning algorithms to predict task execution requirements and device availability. The best offload time relates to the real-time demand of the task. We will grade the real-time demand level of the task. If the task requires real-time response, it may be offloaded to the nearest edge device immediately. Otherwise, we will delay the task offloading and prioritize to select the high real-time task.

Finally, it cannot be ignored that there are complex dependencies in subtasks, which is an essential issue in distributed edge computing task offloading. In this work, we focus on the task offloading strategy in a distributed edge collaborative environment. We currently don't consider it in this work; however, in the future, we will design an efficient and optimal task partitioning to analyze and satisfy the dependencies, such as graph theory [53], [54]. In addition, the task scheduling will also need to consider the dependencies to decide the execution order of subtasks [55].

IX. CONCLUSION

In this article, we propose a heterogeneous distributed EVA framework named EdgeBrain. For the video analytics on the PCO, we consider the remaining computing resources, battery, and bandwidth of edge devices. We propose a Stackelberg game-based offloading strategy to collaboratively process tasks. Specifically, we prove that the solution of Nash equilibrium in the Stackelberg game for the task offloading strategy exists in EdgeBrain, and the solution is unique. We implemented a realistic prototype for EdgeBrain. The experiments validate the stability and efficiency of the distributed collaborative task offloading strategy. In the future, we will focus on the offloading strategy in the multi-PCO and multi-CCO scenarios. Considering the parallelism of edge distributed systems, task offloading between multi-PCOs and multi-CCOs is a complex game problem, aiming to improve the performance of distributed EVA.

REFERENCES

- [1] Z. Zhaoyun et al., "Application of micro-grid control system in smart park," *J. Eng.*, vol. 2019, no. 16, pp. 3116–3119, 2019.
- [2] A. Ferdowsi, U. Challita, and W. Saad, "Deep learning for reliable mobile edge analytics in intelligent transportation systems: An overview," *IEEE Veh. Technol. Mag.*, vol. 14, no. 1, pp. 62–70, Mar. 2019.
- [3] Z. Shouran, A. Ashari, and T. Priyambodo, "Internet of things (IoT) of smart home: Privacy and security," *Int. J. Comput. Appl.*, vol. 182, no. 39, pp. 3–8, 2019.
- [4] Statista, "Number of Internet of Things (IoT) connected devices worldwide in 2018, 2025 and 2030(in billions)." 2019. Accessed: May 14, 2020. [Online]. Available: <https://www.statista.com>
- [5] Y. C. Hu, M. Patel, D. Sabella, N. Sprecher, and V. Young, "Mobile edge computing—A key technology towards 5G," *ETSI White Paper*, vol. 11, no. 11, pp. 1–16, 2015.
- [6] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, Fourth Quarter 2017.
- [7] M. ETSI, "Mobile edge computing (MEC); framework and reference architecture," *ETSI, DGS MEC*, vol. 3, 2016. Accessed: Apr. 4, 2020. [Online]. Available: https://www.etsi.org/deliver/etsi/MEC/001_099/003/01.01.01_60/gsmec003v010101p.pdf
- [8] Z. Yu, Y. Gong, S. Gong, and Y. Guo, "Joint task offloading and resource allocation in UAV-enabled mobile edge computing," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3147–3159, Apr. 2020.
- [9] C.-F. Liu, M. Bennis, and H. V. Poor, "Latency and reliability-aware task offloading and resource allocation for mobile edge computing," in *Proc. IEEE Globecom Workshops*, 2017, pp. 1–7.
- [10] Y. Wang, M. Sheng, X. Wang, L. Wang, and J. Li, "Mobile-edge computing: Partial computation offloading using dynamic voltage scaling," *IEEE Trans. Commun.*, vol. 64, no. 10, pp. 4268–4282, Oct. 2016.
- [11] Y. Wu, J. Chen, L. P. Qian, J. Huang, and X. S. Shen, "Energy-aware cooperative traffic offloading via device-to-device cooperations: An analytical approach," *IEEE Trans. Mobile Comput.*, vol. 16, no. 1, pp. 97–114, Jan. 2017.
- [12] E. Cuervo, A. Balasubramanian, D. K. Cho, A. Wolman, and P. Bahl, "MAUI: Making smartphones last longer with code offload," in *Proc. Int. Conf. Mobile Syst. Appl. Serv.*, 2010, pp. 49–62.
- [13] L. Ma, "Study on intelligent mine based on the application of 5G wireless communication system," in *Proc. IOP Conf. Ser. Earth Environ. Sci.*, 2020, Art. no. 032050.
- [14] Y. Xu et al., "Multi-sensor edge computing architecture for identification of failures short-circuits in wind turbine generators," *Appl. Soft Comput.*, vol. 101, 2021, Art. no. 107053.
- [15] D. Chatzopoulos, C. Bermejo, E. ul Haq, Y. Li, and P. Hui, "D2D task offloading: A dataset-based Q&A," *IEEE Commun. Mag.*, vol. 57, no. 2, pp. 102–107, Feb. 2019.
- [16] G. Li and J. Cai, "An online incentive mechanism for collaborative task offloading in mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 624–636, Jan. 2020.
- [17] Y. He, J. Ren, G. Yu, and Y. Cai, "D2D communications meet mobile edge computing for enhanced computation capacity in cellular networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 3, pp. 1750–1763, Mar. 2019.
- [18] C. Li et al., "Dynamic offloading for multiuser multi-CAP MEC networks: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 70, no. 3, pp. 2922–2927, Mar. 2021.
- [19] Y. Jie, X. Tang, K.-K. R. Choo, S. Su, M. Li, and C. Guo, "Online task scheduling for edge computing based on repeated Stackelberg game," *J. Parallel Distrib. Comput.*, vol. 122, pp. 159–172, 2018.
- [20] G. Mitsis, E. E. Tsiropoulou, and S. Papavassiliou, "Price and risk awareness for data offloading decision-making in edge computing systems," *IEEE Syst. J.*, vol. 16, no. 4, pp. 6546–6557, Dec. 2022.
- [21] M. Chen and Y. Hao, "Task offloading for mobile edge computing in software defined ultra-dense network," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 587–597, Mar. 2018.
- [22] O. Munoz, A. Pascual-Iserte, and J. Vidal, "Optimization of radio and computational resources for energy efficiency in latency-constrained application offloading," *IEEE Trans. Veh. Technol.*, vol. 64, no. 10, pp. 4738–4755, Oct. 2015.
- [23] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [24] K. Zhang et al., "Energy-efficient offloading for mobile edge computing in 5G heterogeneous networks," *IEEE Access*, vol. 4, pp. 5896–5907, 2016.
- [25] M. Satyanarayanan, P. Bahl, R. Caceres, and N. Davies, "The case for VM-based cloudlets in mobile computing," *IEEE Pervasive Comput.*, vol. 8, no. 4, pp. 14–23, Fourth Quarter 2009.
- [26] G. Zhang, W. Zhang, Y. Cao, D. Li, and L. Wang, "Energy-delay tradeoff for dynamic offloading in mobile-edge computing system with energy harvesting devices," *IEEE Trans. Ind. Inform.*, vol. 14, no. 10, pp. 4642–4655, Oct. 2018.
- [27] X. Chen, Y. Cai, Q. Shi, M. Zhao, B. Champagne, and L. Hanzo, "Efficient resource allocation for relay-assisted computation offloading in mobile-edge computing," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 2452–2468, Mar. 2020.
- [28] A. Kiani and N. Ansari, "Toward hierarchical mobile edge computing: An auction-based profit maximization approach," *IEEE Internet Things J.*, vol. 4, no. 6, pp. 2082–2091, Dec. 2017.
- [29] X. Xiong, K. Zheng, L. Lei, and L. Hou, "Resource allocation based on deep reinforcement learning in IoT edge computing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1133–1146, Jun. 2020.
- [30] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic computation offloading for mobile-edge computing with energy harvesting devices," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3590–3605, Dec. 2016.
- [31] R. Yadav, W. Zhang, O. Kaiwartya, H. Song, and S. Yu, "Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14 198–14 211, Dec. 2020.
- [32] R. Yadav et al., "Smart healthcare: RL-based task offloading scheme for edge-enabled sensor networks," *IEEE Sensors J.*, vol. 21, no. 22, pp. 24 910–24 918, Nov. 2021.
- [33] W. Zhang, R. Yadav, Y.-C. Tian, S. K. S. Tyagi, I. A. Elgendy, and O. Kaiwartya, "Two-phase industrial manufacturing service management for energy efficiency of data centers," *IEEE Trans. Ind. Inform.*, vol. 18, no. 11, pp. 7525–7536, Nov. 2022.
- [34] W. Sun, J. Liu, Y. Yue, and H. Zhang, "Double auction-based resource allocation for mobile edge computing in industrial Internet of Things," *IEEE Trans. Ind. Inform.*, vol. 14, no. 10, pp. 4692–4701, Oct. 2018.
- [35] C. Zhao, Y. Cai, A. Liu, M. Zhao, and L. Hanzo, "Mobile edge computing meets mmWave communications: Joint beamforming and resource allocation for system delay minimization," *IEEE Trans. Wireless Commun.*, vol. 19, no. 4, pp. 2382–2396, Apr. 2020.
- [36] Y. Liu, C. Xu, Y. Zhan, Z. Liu, J. Guan, and H. Zhang, "Incentive mechanism for computation offloading using edge computing: A Stackelberg game approach," *Comput. Netw.*, vol. 129, pp. 399–409, 2017.
- [37] S. Jošilo and G. Dán, "Wireless and computing resource allocation for selfish computation offloading in edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 2467–2475.
- [38] X. Deng, Y. Jiang, L. T. Yang, M. Lin, L. Yi, and M. Wang, "Data fusion based coverage optimization in heterogeneous sensor networks: A survey," *Inf. Fusion*, vol. 52, pp. 90–105, 2019.
- [39] T. Zhao, S. Zhou, X. Guo, Y. Zhao, and Z. Niu, "A cooperative scheduling scheme of local cloud and internet cloud for delay-aware mobile cloud computing," in *Proc. IEEE Globecom Workshops*, 2015, pp. 1–6.

- [40] K. Lee and I. Shin, "User mobility-aware decision making for mobile computation offloading," in *Proc. IEEE Conf. Cyber-Phys. Syst. Netw. Appl.*, 2013, pp. 116–119.
- [41] X. Zhao, Y. Shi, and S. Chen, "TS-SMOSA: A multi-objective optimization method for task scheduling in mobile edge computing," *J. Internet Technol.*, vol. 20, no. 4, pp. 1057–1068, 2019.
- [42] Y. Shi, S. Chen, and X. Xu, "MAGA: A mobility-aware computation offloading decision for distributed mobile cloud computing," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 164–174, Feb. 2018.
- [43] Z. LLC, "Zabbix professional services," [EB/OL]. 2001. Accessed: Apr. 4, 2020. [Online]. Available: <https://www.zabbix.com/services>
- [44] X. Lin, J. Li, W. Yang, J. Wu, Z. Zong, and X. Wang, "Vehicle-to-cloudlet: Game-based computation demand response for mobile edge computing through vehicles," in *Proc. IEEE 89th Veh. Technol. Conf.*, 2019, pp. 1–6.
- [45] L. Li, T. Q. Quek, J. Ren, H. H. Yang, Z. Chen, and Y. Zhang, "An incentive-aware job offloading control framework for multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 20, no. 1, pp. 63–75, Jan. 2021.
- [46] Y. Mao, J. Zhang, S. Song, and K. B. Letaief, "Stochastic joint radio and computational resource management for multi-user mobile-edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 16, no. 9, pp. 5994–6009, Sep. 2017.
- [47] C. Tang and H. Wu, "Optimal computational resource pricing in vehicular edge computing: A Stackelberg game approach," *J. Syst. Architecture*, vol. 121, 2021, Art. no. 102331.
- [48] J. Moura and D. Hutchison, "Game theory for multi-access edge computing: Survey, use cases, and future trends," *IEEE Commun. Surv. Tuts.*, vol. 21, no. 1, pp. 260–288, First Quarter 2019.
- [49] E. Voit, "Enterprise network availability: How to calculate and improve," 2020. Accessed: Mar. 7, 2020. [Online]. Available: <https://blogs.cisco.com/networking/enterprise-network-availability-how-to-calculate-and-improve>
- [50] X. Deng, B. Wang, W. Liu, and L. T. Yang, "Sensor scheduling for multi-modal confident information coverage in sensor networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 3, pp. 902–913, Mar. 2015.
- [51] Y. Liu, H. Wu, Y. Xia, Y. Wang, F. Li, and P. Yang, "Optimal online data dissemination for resource constrained mobile opportunistic networks," *IEEE Trans. Veh. Technol.*, vol. 66, no. 6, pp. 5301–5315, Jun. 2017.
- [52] U. Saleem, Y. Liu, S. Jangsher, X. Tao, and Y. Li, "Latency minimization for D2D-enabled partial computation offloading in mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 4, pp. 4472–4486, Apr. 2020.
- [53] L. Chen, J. Wu, J. Zhang, H.-N. Dai, X. Long, and M. Yao, "Dependency-aware computation offloading for mobile edge computing with edge-cloud cooperation," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2451–2468, Fourth Quarter 2022.
- [54] G. Zhao, H. Xu, Y. Zhao, C. Qiao, and L. Huang, "Offloading tasks with dependency and service caching in mobile edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 11, pp. 2777–2792, Nov. 2021.
- [55] Y. Liu et al., "Dependency-aware task scheduling in vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4961–4971, Jun. 2020.



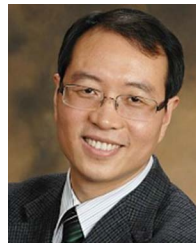
Hui Sun received the PhD degree from Huazhong University Science and Technology, in 2014. He is an associate professor of computer science with Anhui University. His research interests include computer systems and edge computing.



Zhiyong Wang is currently working toward the MS degree with Anhui University. His main research interests include edge computing and intelligent video analytics.



Ying Yu is currently working toward the MS degree with Anhui University. Her main research interest include edge computing.



Kewei Sha (Senior Member, IEEE) received the PhD degree in computer science from Wayne State University, in 2008. He is an associate professor with the Department of Information Sciences, University of North Texas. His research interests include Internet of Things, cyber-physical systems, edge computing, security and privacy, and data management and analytics. His research has been supported by NSF, NASA, UHCL and OCU. He has served as an associate editor of the *IEEE Internet of Things Journal*, *Elsevier Smart Health* and *Springer Computing*, a guest editor at several prestigious international journals, and an organizing committee member of many conferences. He is a senior member of the ACM.



Yalong Wu received the doctor of science degree from Towson University, in 2020. He is an assistant professor of data science with the Department of Computing Sciences, University of Houston-Clear Lake, Texas, USA. His research interests reside in data science, security, and Internet of Things.